

Assignment Submission: Diligencify Profile Agent

Hi everyone, I'm excited to walk you through my submission for the **Diligencify Profile Agent**.

My goal was to build a highly robust, production-ready full-stack application that takes an individual's name and fragmented context clues, and uses a multi-agent AI pipeline to research, verify, and extract a centralized due-diligence intelligence profile.

Whenever I approach a project, I don't just start typing code. I use a **Threat Modeling Approach**. I sit down and map out the biggest points of failure first. For an automated AI identity research tool, I identified three major threats:

1. **The Identity Mix-up Threat:** The AI scraping a web page about "John Smith the Plumber" and merging his data into the profile for "John Smith the CEO".
2. **The Prompt Injection Threat:** The AI scraping a malicious website that contains invisible text saying, *"Ignore previous instructions"*, breaking the JSON pipeline.
3. **The API Fragility Threat:** Free-tier AI rate limits crashing our complex pipeline halfway through processing.

By identifying these threats first, it completely dictated my architectural choices. Here is how that architecture maps to the evaluation criteria:

1. End-to-end AI Workflow and Engineering Approach

The Threat: API Fragility crashing the application mid-generation.

The Solution: We engineered a highly resilient backend capable of "self-healing."

Building this wasn't perfectly smooth. Our biggest roadblock was **Model Selection and Rate Limits**. Initially, I started from gemini 2.5 flash then llama 3b and at last I attempted to use Groq for its lightning-fast speeds, but their strict 6,000 Token-Per-Minute limit instantly crashed on our massive 80,000-character RAG payloads.

I again pivoted to models with massive context windows (like Google Gemini 1.5 Flash), utilizing OpenRouter. However, OpenRouter's globally shared free-tier servers were constantly saturated, returning continuous **429 Too Many Requests** errors.

To solve this, I built an **Aggressive Exponential Backoff** system. I intercepted the standard OpenAI Python client and injected a custom 10-retry loop. If the backend detects a rate limit, it doesn't crash. It silently puts the task to sleep for 35 seconds to let the global limits reset, and tries again. I also dynamically bumped the global backend `asyncio` timeouts to 10 minutes. This created a bulletproof pipeline that successfully pushes massive data payloads through highly unstable free-tier APIs without the user ever seeing a crash screen.

2. Appropriate use of RAG, Agents, Orchestration, or Tool Calling

Our system uses a strictly orchestrated **Two-Agent RAG Pipeline**:

- **The Brainstorming Phase (Researcher Agent):** It dynamically generates hyper-specific search queries based on the target name and known context anchors.
- **The Retrieval Phase:** It fetches raw data using the Tavily Search API.
- **The Synthesis Phase (Extractor Agent):** To solve the **Prompt Injection Threat**, the Extractor Agent wraps the verified 80,000 characters of scraped research inside strict `<source_content>` XML tags. I explicitly instructed the LLM to treat this block strictly as untrusted data strings, effectively neutralizing prompt injections.

3. Accuracy and Structure of the Generated Profile

When processing the final extraction, the AI kept trying to be *too* helpful. Instead of leaving the `sources_master_list` array empty for the backend to fill out with exact RAG metadata (URLs, timestamps), the AI hallucinated the keys, attempting to write the URLs itself. This caused our strict Pydantic validation to instantly crash.

The Pivot: I implemented a mid-stream interceptor. Right before the LLM's raw string is passed to `DiligenceProfile.model_validate()`, our backend intercepts the JSON dictionary and forcefully clears out the hallucinated array (`profile_data["sources_master_list"] = []`), allowing our secure backend code to precisely attach the pristine source data directly.

4. Quality of Public-Source Retrieval and References

To ensure high-quality data retrieval, I built a **Fuzzy Matching Deduplication** engine. When the Researcher pulls URLs from Tavily, it runs a RapidFuzz algorithm (`fuzz.partial_ratio`) to detect if multiple URLs are returning the exact same boilerplate text. If the similarity is above 75%, it drops the duplicate source. This prevents our massive context window from being flooded with redundant data, saving precious token limits.

5. Handling of Missing or Conflicting Information

The Threat: The Identity Mix-up Threat.

The Solution: We built a strict **Disambiguation Engine**.

I passed every single scraped website through a verification layer. An LLM acts as a judge, scoring the web page from 0.0 to 1.0 based on how well it matches our known context anchors (like a specific employer or location). If a source scores below 0.5 (e.g., it refers to a different person with the same name), it is ruthlessly dropped from the context window with an exclusion note. If information is genuinely missing, the strict Pydantic schemas enforce the AI to output exactly "Not publicly available" rather than hallucinating facts.

6. Code Quality and Modularity

The application is strictly modular, dividing responsibilities between the backend (FastAPI) and frontend (Next.js):

- **Agents** (**researcher.py**, **extractor.py**): Encapsulated logic for LLM communication and logic parsing.
- **Services** (**orchestrator.py**, **retrieval.py**): Handles task backgrounding, timeouts, and HTML stripping (using **html2text** for clean markdown conversion).
- **Models** (**profile.py**): Centralized Pydantic schemas that enforce the exact shape of the output JSON.

7. Documentation and Reproducibility

The entire repository is documented in a robust **README.md** that perfectly mirrors production-grade enterprise repositories. It clearly outlines the environment variables needed, the multi-agent application flow, and the exact bash commands to spin up the FastAPI backend and Next.js frontend locally.

Ultimately, by leading with a threat-modeling mindset, I did not just build a wrapper around an AI API. But built a resilient, self-healing enterprise-grade extraction pipeline.

Thank you for your time, and I'd love to hear your feedback on the project!